✅ STEP 1: Set Git account (for push/pull)

Use your **personal GitHub (main account)** for Git commands.

Set it globally:

```
git config --global user.name "YourName"
git config --global user.email "your-personal-email@gmail.com"
```

✅ STEP 2: Use SSH for multiple accounts (IMPORTANT 🔥)

This avoids login conflicts.

1. Generate SSH keys for BOTH accounts

```
ssh-keygen -t ed25519 -C "personal-email@gmail.com"
```

Save as:

id_ed25519_personal

Again for Copilot account:

```
ssh-keygen -t ed25519 -C "copilot-email@gmail.com"
```

Save as:

id_ed25519_copilot

---

2. Add keys to SSH config

Open:

~/.ssh/config

Add:

```
# Personal account (for git)
Host github-personal
   HostName github.com
   User git
   IdentityFile ~/.ssh/id_ed25519_personal

# Copilot account
```

```
Host github-copilot
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_copilot
```

---

3. Add SSH keys to GitHub

Go to:

- GitHub → Settings → SSH Keys

Add:

- personal key → personal account
- copilot key → copilot account

✅ **STEP 3: Clone repo using correct account**

👉 For personal account:

```
git clone git@github-personal:username/repo.git
```

👉 Now all push/pull will use PERSONAL account automatically.

✅ **STEP 4: Login Copilot separately in VS Code**

In **VS Code**:

1. Press Ctrl + Shift + P
2. Search → GitHub: Sign In
3. Login using **Copilot Pro account**

👉 This does NOT affect Git CLI.

⚠️ **IMPORTANT LAB TIP**

If your lab blocks browser login:

Use this trick:

gh auth login

OR use **SSH only (best option)** → no login required.

🧠 **Simple Mental Model**

- Git = SSH key → decides **which account pushes code**
- VS Code = Login session → decides **which account gives Copilot**

👉 They are independent.

🐳 **1. Configure Docker Credentials in Jenkins**

👉 **Steps:**

1. Open Jenkins Dashboard
2. Go to:
   Manage Jenkins → Manage Credentials
3. Select:
   - (global) domain
   - Click **Add Credentials**

🔑 **Add Docker Hub Credentials**

- Kind → **Username with password**
- Username → your Docker Hub username
- Password → your Docker Hub password
- ID → docker-creds (IMPORTANT for pipeline)

💡 **Example:**

ID: docker-creds
Username: druv123
Password: ********

✅ **Use in Jenkins Pipeline**

```
pipeline {
  agent any
  stages {
    stage('Docker Login') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'docker-creds',
          usernameVariable: 'DOCKER_USER',
          passwordVariable: 'DOCKER_PASS'
        )]) {
          sh '''
          echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
          '''
        }
      }
    }
  }
}
```

🛞 **2. Configure Kubernetes Credentials in Jenkins**

Now this is the **most important part** for your lab.

🧠 **What Jenkins needs:**

Kubernetes uses a file called:

👉 kubeconfig

📄 **Step 1: Get kubeconfig file**

Run this on your system:

kubectl config view --raw

OR directly use:

~/.kube/config

📌 **Step 2: Add to Jenkins**

1. Go to:
   Manage Jenkins → Manage Credentials
2. Click **Add Credentials**

🔑 **Add Kubernetes Config**

- Kind → **Secret file**
- Upload → your config file
- ID → kubeconfig

💡 **Alternative (also works in exams):**

- Kind → **Secret Text**
- Paste full kubeconfig content
- ID → kubeconfig

🚀 **3. Use Kubernetes Config in Pipeline**

✅ **Example Pipeline:**

```
pipeline {
  agent any
  stages {
    stage('Deploy to Kubernetes') {
```

```
        steps {
            withCredentials([file(
                credentialsId: 'kubeconfig',
                variable: 'KUBECONFIG'
            )]) {
                sh '''
                kubectl get pods
                kubectl apply -f deployment.yaml
                '''
            }
        }
    }
  }
}
```

🔥 **Full DevOps Flow (Exam Gold Answer)**

```
pipeline {
  agent any

  stages {
    stage('Build Image') {
      steps {
        sh 'docker build -t druv/app .'
      }
    }

    stage('Push Image') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'docker-creds',
          usernameVariable: 'USER',
          passwordVariable: 'PASS'
        )]) {
          sh '''
          echo $PASS | docker login -u $USER --password-stdin
          docker push druv/app
```

```
            '''
        }
    }
}

stage('Deploy') {
    steps {
        withCredentials([file(
            credentialsId: 'kubeconfig',
            variable: 'KUBECONFIG'
        )]) {
            sh 'kubectl apply -f deployment.yaml'
        }
    }
}
}
}
```

⚠️ **Common Mistakes (VERY IMPORTANT)**

❌ Wrong credential type

✔ Use:

- Docker → Username/Password
- Kubernetes → Secret File

❌ Not giving correct ID

✔ Always match:

```
credentialsId: 'docker-creds'
```

❌ kubeconfig not valid

✔ Ensure:

```
kubectl get nodes
```

works before adding